NLP COURSE · LECTURE 2

NLP Pipelines

From "what is it?" to "how do pros build it end-to-end" — a complete, visual, dark-mode walkthrough of the five stages every NLP system passes through, with the math you actually need and a real worked example.

1. Data Acquisition
2. Text Preparation
3. Feature Engineering
4. Modeling
5. Deployment

GETTING STARTED

What You'll Learn

This guide turns a spoken lecture into a structured, easy-to-scan reference. It goes from the plain-English basics of what an NLP pipeline even is, all the way to the math and trade-offs a working NLP engineer thinks about every day. Nothing from the original lecture is skipped — it's reorganized, illustrated, and expanded with examples and diagrams so it's easier to absorb.

01 What is an NLP Pipeline? BASIC

02 The Big Picture (5 steps + why it's non-linear) BASIC

03 Step 1 — Data Acquisition BASIC

04 Step 2 — Text Preparation INTERMEDIATE

05 Step 3 — Feature Engineering (with math) ADVANCED

06 Step 4 — Modeling & Evaluation ADVANCED

07 Step 5 — Deployment, Monitoring & Updating PRO

08 Worked Example — Flipkart Sentiment App PRO

09 Assignment — Quora Duplicate Questions PRO

10 Full Pipeline Cheat-Sheet PRO

WHY THIS MATTERS

If you join a company as an NLP engineer, your job is rarely "run an algorithm on a CSV." You're handed a vague goal like "*add sentiment detection to our review page*" — and you're expected to figure out the entire path from raw data to a live, monitored feature. That path is the pipeline.

01 · BASIC

What is an NLP Pipeline?

In one line: an NLP pipeline is the ordered series of steps you follow to go from raw, messy text data all the way to a working, deployed piece of NLP software.

Think of it like a factory assembly line. Raw material (text) enters at one end, passes through a series of stations (cleaning, feature extraction, modeling), and a finished, usable product (a working feature — e.g. a spam filter, a sentiment score, a chatbot reply) comes out the other end.

> **KEY IDEA**
>
> Even the simplest heuristic ("if the review contains the word 'bad', mark it negative") still needs data. **Without data, there is no pipeline at all** — that's why Data Acquisition is always step one.

02 · BASIC

The Big Picture

The 5 Stages, at a Glance

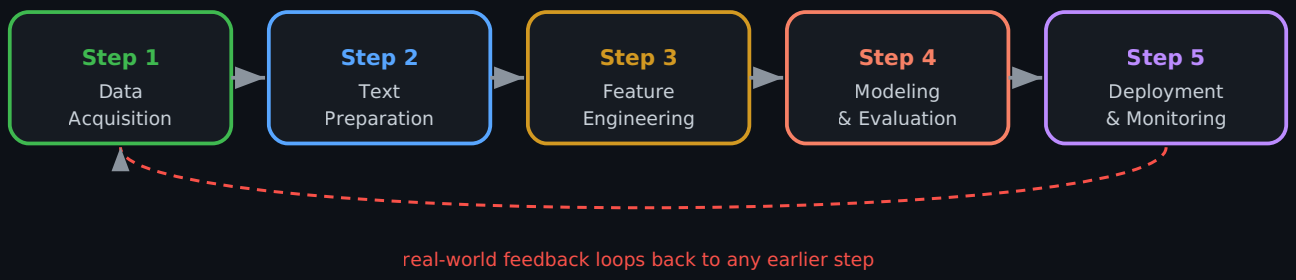


Figure 1 — The 5-step NLP pipeline, with the dashed line showing that it is non-linear.

Two Disclaimers (read these before going further)

DISCLAIMER 1 — IT'S NOT UNIVERSAL

This 5-step shape fits tasks like **text classification** or **sentiment analysis** well. But for **chatbots** or **text summarization** — especially with deep learning — the pipeline looks somewhat different. Treat this as the default template, not a law of physics.

DISCLAIMER 2 — IT'S NON-LINEAR

You almost never go 1 → 2 → 3 → 4 → 5 once and stop. A very common real story: you deploy (step 5), discover the model performs badly on real users, and jump back to Feature Engineering or Modeling to fix it. The pipeline is a loop, not a straight line.

Why Learn This At All?

If you land an NLP engineer role, your work is rarely "just apply an algorithm to a dataset and report accuracy." You'll be handed **end-to-end asks** — build this feature, ship it, keep it working. Without a mental map of the whole pipeline, it's hard to even know what questions to ask first.

03 · BASIC — STEP 1 OF 5

Data Acquisition

Data acquisition is the single most crucial step — no data, no pipeline. Imagine you work at **Flipkart** and are asked to build a **Sentiment Analysis app for customer reviews**. There are three broad situations you could be in.

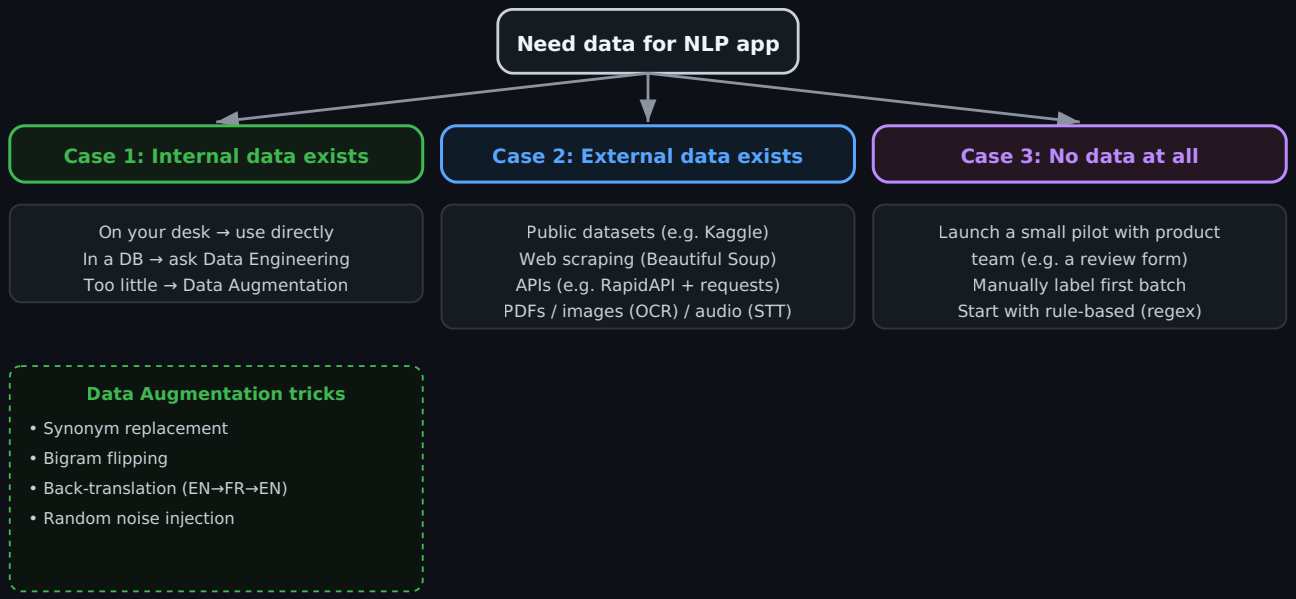


Figure 2 — Decision tree for sourcing your dataset.

Case 1 — Data Available Internally

- **On your desk:** a CSV or similar is already sitting there — jump straight to Text Preparation.
- **In the database:** the data exists but is locked inside company systems — coordinate with the Data Engineering team to extract it.
- **Insufficient data:** you have some, but not enough to train a model. Use **Data Augmentation** — synthetically expanding your dataset via synonym replacement, bigram flipping, back-translation, or adding controlled random noise.

Case 2 — Data Available Externally

- **Public datasets** — platforms like Kaggle, though they may not match your exact use case.
- **Web scraping** — libraries like Beautiful Soup to pull data from competitor sites.
- **APIs** — e.g. pulling from RapidAPI via Python's `requests` library.
- **Non-text sources** — PDFs need parsing libraries, images need OCR, audio needs Speech-to-Text.

Case 3 — No Data Exists Yet

You're building something genuinely new. Work with product/software teams to run a small pilot — for example, a review form shown only to loyal customers. As the first responses trickle in, label them manually. Until you have enough volume for ML, lean on simple **rule-based (regex) heuristics**.

Text Preparation

Raw text is messy. Before any model can use it, you run it through three layers: basic cleaning, basic preprocessing, and (optionally) advanced preprocessing.

Layer 1 — Basic Cleaning

- Remove HTML tags (regular expressions)
- Normalize Unicode — e.g. turn emojis into machine-readable tokens
- Spell-check fast-typing slang / "fat finger" typos (e.g. with TextBlob)

Layer 2 — Basic Preprocessing

- **Tokenization** — split paragraphs into sentences, sentences into words
- **Stop word removal** (optional) — drop low-information words like "and", "or", "the"
- **Stemming & Lemmatization** (optional) — collapse *dance / dancing / danced* to one root
- Remove punctuation and digits
- Lowercasing — for consistency
- Language detection

Layer 3 — Advanced Preprocessing

- **POS Tagging** — label nouns, verbs, adjectives (crucial for chatbots)
- **Parsing** — recover the syntactic structure of a sentence
- **Coreference Resolution** — teach the machine that "Hitesh" and "He" refer to the same entity in a paragraph

ORDER MATTERS

If you strip stop words *before* POS tagging or parsing, those advanced steps will fail — many of them rely on function words like "is", "the", "of" to understand sentence structure.

Worked Example — Watch a Sentence Transform

RAW INPUT

"This product is AMAZING!! Bought it on 12/03, best purchase ever
 highly recommend."

Stage	Operation	Result
Basic cleaning	Strip HTML, normalize emoji	"This product is AMAZING!! [heart-eyes-emoji] Bought it on 12/03, best purchase ever highly recommend."
Basic preprocessing	Lowercase, remove punctuation/digits	"this product is amazing bought it on best purchase ever highly recommend"
Basic preprocessing	Tokenize	[this, product, is, amazing, bought, it, on, best, purchase, ever, highly, recommend]
Basic preprocessing (optional)	Remove stop words	[product, amazing, bought, best, purchase, ever, highly, recommend]
Basic preprocessing (optional)	Lemmatize	[product, amazing, buy, best, purchase, ever, highly, recommend]
Advanced	POS tag (on the pre-stopword-removal version)	this/DET product/NOUN is/VERB amazing/ADJ bought/VERB it/PRON …

05 · ADVANCED — STEP 3 OF 5

Feature Engineering

Machines work on numbers, not words. Feature engineering is the process of converting cleaned text into numeric columns (features) that a model can actually learn from.

The Simplest Possible Feature Set

For sentiment analysis, a very basic hand-built feature set could simply be counts:

Review	# Positive Words	# Negative Words	# Neutral Words
"amazing, best purchase, highly recommend"	3	0	1
"broke in two days, terrible quality"	0	2	2

Bag of Words (BoW)

BoW represents each document as a vector of word counts, ignoring grammar and word order — hence "bag."

VOCABULARY {good, bad, product, phone} · **DOC A** "good product" · **DOC B** "bad product bad phone"

	good	bad	product	phone
Doc A	1	0	1	0
Doc B	0	2	1	1

TF-IDF (Term Frequency – Inverse Document Frequency)

BoW treats every word as equally important — but common words across all documents (like "product") carry less signal than rare, distinctive ones. TF-IDF fixes this by weighting each word by how rare it is *across the whole corpus*.

Term Frequency — how often term t appears in document d

```
TF(t, d) = count(t, d) / total terms in d
```

Inverse Document Frequency — how rare term t is across all N documents

```
IDF(t) = log( N / df(t) )   where df(t) = number of documents containing t
```

Final TF-IDF weight

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

MINI EXAMPLE

Corpus of N = 3 reviews. The word "phone" appears in all 3 reviews (common → low IDF). The word "battery" appears in only 1 review (rare → high IDF). Even if both words appear once in a given review, "battery" gets a higher TF-IDF score because it's more distinctive — it tells you more about that specific review.

Word2Vec — Dense Word Embeddings

Instead of sparse count vectors, Word2Vec learns a short dense vector (e.g. 100–300 numbers) per word such that words with similar meaning end up close together in that vector space.

Similarity between two word vectors, using cosine similarity

$$\text{similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

A value close to `1` means the words are used in very similar contexts (e.g. "king" and "queen"); a value close to `0` means they're unrelated.

ML vs. DL: Two Philosophies of Feature Engineering

Machine Learning approach

You hand-craft features using domain knowledge (word counts, TF-IDF, sentiment lexicons…).

Pros: high interpretability — you can explain exactly why a prediction was made.

Cons: needs strong domain knowledge; bad features quietly ruin the model.

Deep Learning approach

You feed preprocessed text directly into a neural network; it learns its own features (embeddings).

Pros: no manual feature design needed — the network does the heavy lifting.

Cons: a black box — much harder to explain a given prediction.

Modeling & Evaluation

Model Building — Pick Your Approach by Data Volume

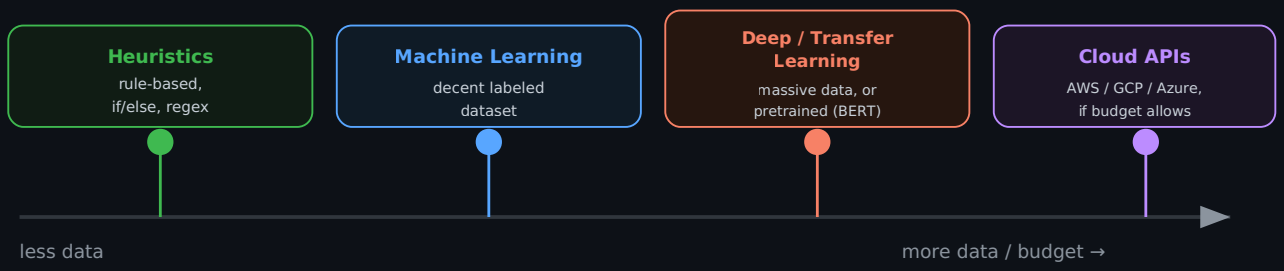


Figure 3 — Four ways to build the model, ordered by how much data (or budget) you have.

1. **Heuristic methods** — for very low data. E.g. "If email is from address X → mark as spam."
2. **Machine Learning** — once you have a decent labeled dataset. Heuristics can even become features inside your ML model.
3. **Deep Learning / Transfer Learning** — for massive datasets, or by reusing pretrained models like **BERT** that were trained on huge internet-scale corpora and fine-tuned for your task.
4. **Cloud APIs** — ready-made NLP endpoints from AWS, Google Cloud, or Azure when you have budget and don't need a custom model.

Evaluation — Two Different Lenses

Intrinsic evaluation

Technical, model-centric metrics: **Accuracy**, **Precision**, **Recall**, **Perplexity**. Measured offline, on a test set.

Extrinsic evaluation

Business-centric, measured after deployment. Example: an autocomplete feature can have great intrinsic accuracy, but if **real users never click the suggestions**, its extrinsic value is zero.

BOTH MATTER

A model can win on paper (intrinsic) and still fail in the real product (extrinsic) — that gap is exactly why the pipeline loops back instead of ending at "deploy."

07 · PRO — STEP 5 OF 5

Deployment, Monitoring & Updating

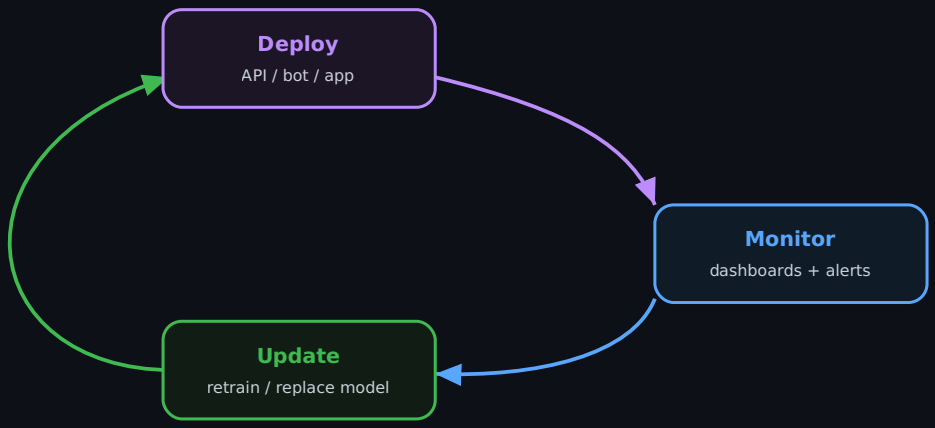


Figure 4 — Deployment is a cycle, not a finish line.

1. Deploying

Depending on the product, this might mean shipping the model as a **microservice / API**, a **WhatsApp bot**, or embedding it directly inside an **Android app**.

2. Monitoring

Build dashboards that track both intrinsic and extrinsic metrics over time, with alerts if performance quietly degrades.

3. Updating

Retrain or replace the model as new data arrives — for example, launching your spam classifier in a new country means encountering new spam patterns it's never seen.

08 · PRO

Worked Example — Flipkart Sentiment App, End to End

Pulling every step together for the running example used throughout this guide:

Step	What happens for the Flipkart sentiment app
1. Data Acquisition	Reviews already sit in Flipkart's database → work with Data Engineering to extract them (Case 1, internal data).
2. Text Preparation	Strip HTML from review widgets, normalize emojis, fix typos, tokenize, lowercase, optionally remove stop words and lemmatize.
3. Feature Engineering	Start simple with TF-IDF vectors; if enough data exists, move to embeddings from a pretrained transformer.
4. Modeling	Try classic ML (Logistic Regression / SVM on TF-IDF) as a baseline, then a fine-tuned BERT model; evaluate with Accuracy/F1 (intrinsic) and click-through / complaint-rate change (extrinsic).
5. Deployment	Ship as an internal API called by the product-review page; monitor prediction distribution and user overrides; retrain quarterly or when a big product category shift occurs.

09 · PRO — YOUR TURN

Assignment — Quora Duplicate Questions

PROBLEM STATEMENT

Quora often gets the same question phrased two different ways — e.g. "*Is IPL destroying Indian cricket?*" vs. "*Is IPL spoiling Indian cricket?*" — and this splits the answers across two separate question pages. Given two questions, predict whether they mean the same thing.

Assume you **do not** work at Quora and have **no internal data**. Map out the full pipeline for this problem. You don't need to write any code — just think it through and write down your reasoning.

1. **Data** — Where will you source labeled pairs of duplicate / non-duplicate questions from?
2. **Text Preparation** — What cleaning, basic preprocessing, and advanced preprocessing will this task need?
3. **Feature Engineering** — What features would actually help a model tell if two questions mean the same thing?
4. **Modeling & Evaluation** — Which modeling approach fits, and which metrics (intrinsic and extrinsic) would you track?
5. **Deployment** — How would you integrate, monitor, and update this feature once it's live?

WHY BOTHER

Working through this by hand — no coding required — is exactly the exercise that builds real-world NLP intuition. Once you can map any new NLP problem onto these five steps, you're no longer guessing.

10 · PRO — REFERENCE

Full Pipeline Cheat-Sheet

Step	Core question it answers	Typical tools / techniques
1. Data Acquisition	Where does the raw text come from?	Internal DB, Kaggle, scraping, APIs, OCR/STT, augmentation
2. Text Preparation	Is the text clean and structured enough to use?	Regex cleaning, tokenization, stopwords, stemming/lemmatization, POS tagging, parsing, coreference
3. Feature Engineering	How do I turn text into numbers?	Word counts, Bag of Words, TF-IDF, Word2Vec / embeddings
4. Modeling	Which algorithm fits my data volume, and is it actually good?	Heuristics, classic ML, DL/Transfer learning (BERT), Cloud APIs · Accuracy/Precision/Recall vs. business metrics
5. Deployment	How does this stay alive and useful in production?	API/bot/app deployment, dashboards + alerting, scheduled retraining

REMEMBER

This template fits classification-style problems best. For chatbots or summarization, expect the shape to shift — and no matter the task, expect to loop backward, not just march forward.